

Chapter 59

BitScrambler

The ESP32-P4 has an extensive amount of DMA-capable peripherals. These can move data from memory to an external device, and vice versa, without any interference from the CPU. This only works if the external device needs or emits the data in question in the same format as the software expects it: if not, the CPU needs to rewrite the format of the data. Examples include a need to swap bytes, reverse bytes, and shift the data left or right.

As bitwise operations tend to be fairly CPU-expensive and the purpose of DMA is to not use the CPU in the transfer, ESP32-P4 includes two BitScramblers, which are dedicated peripherals to change the format of data in between memory and the peripheral. A BitScrambler is capable of performing the aforementioned operations, but as a flexible programmable state machine, it is capable of more advanced things as well. One of the two BitScramblers is dedicated to memory-to-peripheral (or memory-to-memory) transfers, the other one is dedicated to peripheral-to-memory transfers.

59.1 Introduction

In ESP32-P4, a DMA-capable peripheral is one that can read data directly from memory and/or write data directly to memory using a DMA channel. The format this data is read/written in is dependent on the peripheral and generally has only a small amount of flexibility. For most DMA-capable peripherals in ESP32-P4, it is possible to attach a BitScrambler to any of these paths for additional flexibility.

When a BitScrambler is attached to a peripheral, it gets the chance to modify any data that flows down a DMA channel associated with that peripheral. Data written to the DMA channel will appear on the BitScrambler input, and what the BitScrambler writes to its output will flow down the DMA channel. The BitScrambler contains instruction memory where instructions on how to route the data between input and output can be stored. As such, by writing an appropriate BitScrambler program, the way the data is rewritten by the BitScrambler can be controlled.

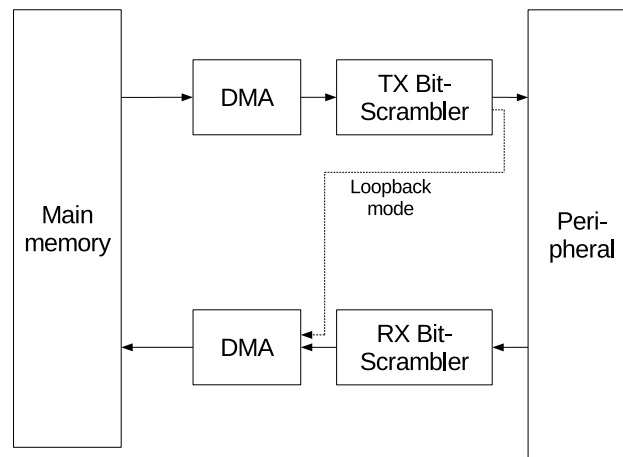


Figure 59.1-1. BitScrambler System Context Diagram

59.2 Feature List

The BitScrambler has the following features:

- Two BitScramblers, one for RX (peripheral-to-memory), one for TX (memory-to-peripheral)
- Support for memory-to-memory transfers
- Processing up to 32 bits per DMA clock period
- Data path controlled by a BitScrambler program stored in instruction memory
- Input registers able to read 0, 8, 16, or 32 bits per clock cycle
- Output registers:
 - Able to write 0, 8, 16, or 32 bits per clock cycle
 - Data sources for output register bits: 64 bits of input data, two counters, LUT RAM data, data output of last cycle, comparators
 - With some restrictions, each of the 32 output register bits can come from any bit on the data sources
- 8 x 257-bit instruction memory, for storing eight instructions, controlling control flow and the data path
- 2048 bytes of lookup table (LUT) memory, configurable as various word widths

59.3 Architectural Overview

The BitScrambler can be separated into a data path, a control path, and the registers that the CPU can use to program them. The data path moves bits between the incoming DMA stream, internal registers, and the outgoing DMA stream. The control path parses the instructions in the instruction memory to control the data path, as well as handle program flow.

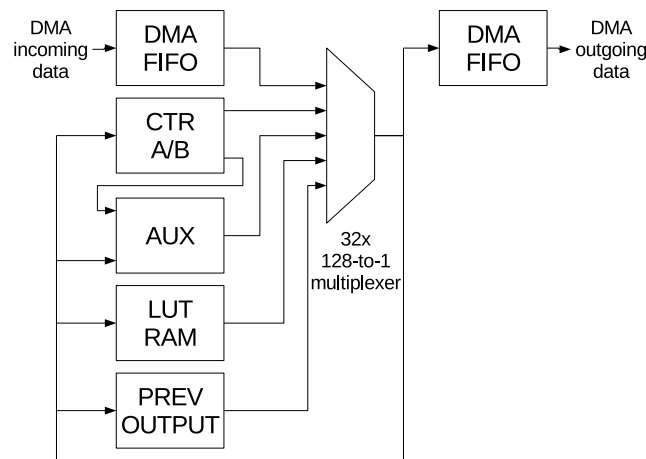


Figure 59.3-1. BitScrambler Data Path Diagram

59.3.1 Data Path

The data path of the BitScrambler is intended to take data from the incoming DMA stream, process it according to the program code stored in instruction memory, and write it to the outgoing DMA stream. Data is read from the incoming DMA stream into a 64-bit register. At the end of an instruction cycle, as specified in the instruction, N bits (with N being 0, 8, 16, or 32) are read from the DMA stream. The data in the register is then shifted toward the LSB of the register with the least significant N bits disappearing. Finally, the read data appears as the N most significant bits. Optionally, on startup, the BitScrambler will automatically read the first 64 bits into this register.

On the other side of the BitScrambler, data is deposited into a 32-bit output register. At the end of the instruction cycle, depending on what the instruction specifies, the least significant 0, 8, 16, or 32 bits will be written to the outgoing DMA stream.

The BitScrambler derives its name from the fact that it can put input bits in any random position in the output, and this is achieved by 32 x 128-to-1 multiplexers. For each of the 32 output register bits, there is one multiplexer that selects the input signals from one of the data sources, dependent on the output of the BitScrambler control path. These data sources are:

- 64 bits, shifted in from the input FIFO (i.e., the left DMA FIFO block in Figure 59.3-1). As described before, data from the incoming DMA stream is deposited here. In order to facilitate iterating over input bits in a loop, an instruction can enable relative addressing. In this addressing mode, any mux sourcing a bit from the input FIFO register will actually get the bit offset by the counter A register (i.e., CTR A in Figure 59.3-1).
- 32 bits that were sent to the output in the last cycle (i.e., PREV OUTPUT block in Figure 59.3-1). Data sent to the output register will appear on this register one clock cycle later. This allows the BitScrambler to 'remember' data over multiple clock cycles: by outputting a bit to the output register, even if that bit subsequently is not sent to the output FIFO, the next clock cycle can still access it by selecting from this register. Bits can be remembered longer-term by selecting bits from this register for output again, which makes them appear here in the subsequent cycle.
- 8, 16, or 32 bits that are the output of LUT RAM. The address used to select the data is bit 16 to 24, 25 or 26 of the output data of the last cycle, assuming the LUT width is 32, 16 or 8 bits, respectively. The LUT RAM is a part of the BitScrambler that takes an address, looks it up in its memory, and outputs the data

located at that specific address. The LUT RAM can be filled when the BitScrambler is initialized, but the BitScrambler itself does not have the ability to modify it. Depending on requirements, the LUT RAM can be initialized in the following modes:

- 512 x 32-bit words
- 1024 x 16-bit words
- 2048 x 8-bit words

The LUT RAM can be used to do translations of data, e.g. using a calibration curve for a given sensor. It can also be used to implement mathematical functions, with the address input split into two or more input variables and the data output used as the output of the function; the memory should be loaded with the output of the function for each of the input values.

- 32 bits from 2 x 16-bit counters (i.e., CTR A/B block in Figure 59.3-1). This register contains two 16-bit counters, named 'A' and 'B'. These can be loaded, incremented and decremented via control path instructions.
- 30 bits from comparators between counter B and the previous data on the output (PREV OUT in Figure 59.3-1; the 30 bits described here are part of the AUX block there). These allow the result of comparisons being used for output or for program flow. This can be useful in e.g. decoding run-length encoded streams, or generating PWM signals.
- 2 bits, one fixed-high, one fixed-low (i.e., part of AUX in Figure 59.3-1). These are useful if a protocol needs an always-high or always-low bit in the output that does not depend on the input data.

Note that some sources cannot be accessed at the same time. Specifically, a given instruction cannot source data from both the high 32 bits of the input FIFO source register as well as the counter registers. When data is read from the LUT, for a LUT that is set to a width of N, the top N bits of both the input FIFO register as well as the top N bits of the counter register become unavailable.

59.3.2 Control Path

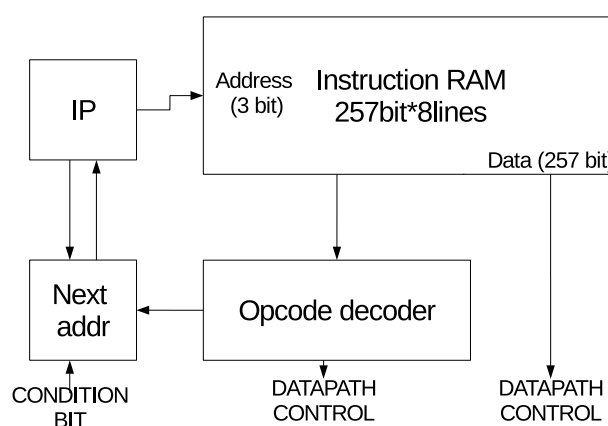


Figure 59.3-2. BitScrambler Control Path Diagram

The BitScrambler behavior is governed by a program stored in program RAM. The program RAM has space for up to eight instructions, each being 257 bits in size. An instruction configures the data path for the clock cycle the instruction runs in, as well as contains opcodes that control program flow.

A BitScrambler program is similar to a microprocessor program, in that each cycle the BitScrambler will execute the next instruction in instruction memory, unless the opcode tells it to specifically jump to another location. Concretely, this behaviour is implemented using an instruction pointer (IP, see Figure 59.3-2), which points to the currently executing instruction. Opcodes can also affect the counter registers.

The bits of the instruction that configure the data path consist of settings for the 32 muxes, as well as some bits to select the availability of the counter bits or LUT output. It also can put the muxes into relative mode, where if they get a bit from the DMA FIFO, the position of that bit is offset by counter B.

59.4 Functional Description

The BitScrambler is a flexible device, with its behavior controlled by the program loaded into it as well as the configuration registers that set several global BitScrambler behaviors as well. As such, a BitScrambler program consists of the contents of the 257-bit x 8 instruction RAM as well as the settings for various configuration registers. Both are described below.

59.4.1 Instructions

The 257 bits that make up an instruction are structured as shown in Table 59.4-1.

Table 59.4-1. Instruction Structure

Bit	Field	Bit	Field
0 - 6	CTL_MUX_SEL_0	133 - 139	CTL_MUX_SEL_19
7 - 13	CTL_MUX_SEL_1	140 - 146	CTL_MUX_SEL_20
14 - 20	CTL_MUX_SEL_2	147 - 153	CTL_MUX_SEL_21
21 - 27	CTL_MUX_SEL_3	154 - 160	CTL_MUX_SEL_22
28 - 34	CTL_MUX_SEL_4	161 - 167	CTL_MUX_SEL_23
35 - 41	CTL_MUX_SEL_5	168 - 174	CTL_MUX_SEL_24
42 - 48	CTL_MUX_SEL_6	175 - 181	CTL_MUX_SEL_25
49 - 55	CTL_MUX_SEL_7	182 - 188	CTL_MUX_SEL_26
56 - 62	CTL_MUX_SEL_8	189 - 195	CTL_MUX_SEL_27
63 - 69	CTL_MUX_SEL_9	196 - 202	CTL_MUX_SEL_28
70 - 76	CTL_MUX_SEL_10	203 - 209	CTL_MUX_SEL_29
77 - 83	CTL_MUX_SEL_11	210 - 216	CTL_MUX_SEL_30
84 - 90	CTL_MUX_SEL_12	217 - 223	CTL_MUX_SEL_31
91 - 97	CTL_MUX_SEL_13	224 - 249	OPCODE
98 - 104	CTL_MUX_SEL_14	250 - 251	CTL_READ_IN
105 - 111	CTL_MUX_SEL_15	252 - 253	CTL_WR_OUT
112 - 118	CTL_MUX_SEL_16	254	CTL_MUX_REL
119 - 125	CTL_MUX_SEL_17	255	CTL_CTR_SRC_SEL
126 - 132	CTL_MUX_SEL_18	256	CTL_LUT_SRC_SEL

The meanings of these fields are as follows:

- CTL_MUX_SEL_n: This 7-bit field selects the source of bit n in the output register. See Table 59.4-3 for details.

- **OPCODE:** These bits encode an opcode. For the bit pattern of this field, please refer to Table 59.4-4.
- **CTL_READ_IN:** This selects the number of bits that are read from the input FIFO at the end of the instruction:
 - 0: Do not read any data
 - 1: Read 8 bits
 - 2: Read 16 bits
 - 3: Read 32 bits
- **CTL_WR_OUT:** This selects the number of bits that are written from the output register to the output FIFO at the end of the instruction:
 - 0: Do not write any data
 - 1: Write 8 bits
 - 2: Write 16 bits
 - 3: Write 32 bits
- **CTL_MUX_REL, CTL_CTR_SRC_SEL and CTR_LUT_SRC_SEL:** Select which bits are available to select using the CTL_MUX_SEL bits.
 - **CTL_CTR_SRC_SEL** (Control Counter Source Select) makes the bits from counter A and B available
 - **CTL_LUT_SRC_SEL** (Control LUT Source Select) makes the output value of the LUT RAM available. See Table 59.4-3 for details.
 - When **CTL_MUX_REL** (Control Mux Relative) is 1, the BitScrambler applies the value of counter A as an offset to any CTL_MUX_SEL_n value that is 63 or below. The exact behaviour depends on CTR_SRC_SEL, as detailed in Table 59.4-2.

Table 59.4-2. Effective CTL_MUX_SEL_n values when CTL_MUX_REL is 1

Condition	Effective value of CTL_MUX_SEL
CTL_MUX_SEL_n ≥ 64	CTL_MUX_SEL (unchanged)
CTL_MUX_SEL_n < 64 and CTL_CTR_SRC_SEL = 0	(CTL_MUX_SEL_n + CTRA) & 63
CTL_MUX_SEL_n < 31 and CTL_CTR_SRC_SEL = 1	(CTL_MUX_SEL_n + CTRA) & 31
32 ≤ CTL_MUX_SEL_n < 64 and CTL_CTR_SRC_SEL = 1	((CTL_MUX_SEL_n + CTRA) & 31) + 32

Of these fields, most are data path configuration items. The field that affects the control path is OPCODE, although CTL_MUX_REL, CTL_CTR_SRC_SEL, and CTR_LUT_SRC_SEL may influence how some of the opcodes work by adjusting the source selection for the IF/IFN opcodes.

CTL_MUX_SEL_n: These fields, one for each bit in the output register, select the source for that particular bit. The specific bit chosen is partially dependent on the settings of the CTL_MUX_REL, CTL_CTR_SRC_SEL and CTR_LUT_SRC_SEL. Given CTL_MUX_SRC_n has value N, the bit is selected as described in Table 59.4-3. Note that the source selection for IF/IFN opcodes is affected in the same way.

Table 59.4-3. CTL_MUX_SEL Values

Bit	Description
0-31	The bit is sourced from bit N in the incoming FIFO register.
32-63	The source of the bit depends on the setting of CTL_CTR_SRC_SEL and CTL_LUT_SRC_SEL: If CTL_CTR_SRC_SEL = 0: the bit is sourced from bit N in the incoming FIFO register. If CTL_CTR_SRC_SEL = 1: the bit is sourced from bit (N-32) of the counter registers. Specifically, if N is 32 to 47, the bit is sourced from bit (N-32) of counter A and if N is 48 to 63, the bit is sourced from bit (N-48) of counter B. If CTL_LUT_SRC_SEL = 1: Depending on the width of the LUT (8, 16, or 32 bits), a read of the top 8, 16, or 32 bits will return data from the LUT rather than what CTL_CTR_SRC_SEL selects. Specifically, given a LUT width of N, 63 will return LUT output bit (N-1), 62 will return LUT output bit (N-2) and so on.
64	CounterB[7:0] =< last[7:0]
65	CounterB[7:0] > last[7:0]
66	CounterB[7:0] = last[7:0]
67	CounterB[7:0] =< last[15:8]
68	CounterB[7:0] > last[15:8]
69	CounterB[7:0] = last[15:8]
70	CounterB[7:0] =< last[23:16]
71	CounterB[7:0] > last[23:16]
72	CounterB[7:0] = last[23:16]
73	CounterB[7:0] =< last[31:24]
74	CounterB[7:0] > last[31:24]
75	CounterB[7:0] = last[31:24]
76	CounterB[15:8] =< last[7:0]
77	CounterB[15:8] > last[7:0]
78	CounterB[15:8] = last[7:0]
79	CounterB[15:8] =< last[15:8]
80	CounterB[15:8] > last[15:8]
81	CounterB[15:8] = last[15:8]
82	CounterB[15:8] =< last[23:16]
83	CounterB[15:8] > last[23:16]
84	CounterB[15:8] = last[23:16]
85	CounterB[15:8] =< last[31:24]
86	CounterB[15:8] > last[31:24]
87	CounterB[15:8] = last[31:24]
88	CounterB[15:0] =< last[15:0]
89	CounterB[15:0] > last[15:0]
90	CounterB[15:0] = last[15:0]
91	CounterB[15:0] =< last[31:16]
92	CounterB[15:0] > last[31:16]
93	CounterB[15:0] = last[31:16]
94	Always 0
95	Always 1
96-127	The bits selected here are the same as the bits on the output register at the end of the previous cycle.

The BitScrambler recognizes six opcodes, which are encoded in the opcode fields as shown in Table 59.4-4.

Table 59.4-4. Instruction Opcode

Bit	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LOOP	1	c	tgt			end_val																ctr_add				
ADD	0	c	h	l	0	0	0	0	0	0	ctr_add															
IF	0	0	tgt			0	0	0	0	1	0	0	0	0	0	0	0	0	0	src						
IFN	0	0	tgt			0	0	0	1	0	0	0	0	0	0	0	0	0	0	src						
LDCTD	0	c	h	l	0	0	0	0	1	1	ctr_set															
LDCTI	0	c	h	l	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

The fields are defined as follows:

- c: Configures which counter to use.
 - 0: counter A
 - 1: counter B
- end_val, ctr_add, ctr_set: 16-bit (or 5-bit) value. Please refer to the instruction descriptions below for the meanings of these fields.
- tgt: The location of an instruction. Range: 0 ~ 7.
- h: 1 if the upper 8 bits need to be written back to a counter.
- l: 1 if the lower 8 bits need to be written back to a counter.

Note that if an opcode does not specify H or L, it is assumed that the full 16 bits need to be written back and the h and l bits will both be 1.

These six opcodes affect the state of the BitScrambler as follows:

Table 59.4-5. LOOP Opcode

LOOPc end_val, ctr_add, tgt	
Loop on counter indicated by c, add ctr_add to the counter, and loop back to tgt until the counter reaches end_val. When that happens, reset the counter and fall through.	
Argument	c: Either A or B, indicates counter to use tgt: 3-bit target instruction addresses end_val: 16-bit value to compare the counter to ctr_add: 5-bit signed value to add to the counter
Pseudo-code	<pre> if ((unsigned)ctrC < end_val) begin IP <= tgt ctr_c <= ctr_c + sign_extend(ctr_add) end else begin IP <= IP + 1 ctr_c <= 0 end </pre>

Table 59.4-6. ADD Opcode

ADDc[H L] ctr_add	
Add immediate value to counter indicated by c, and optionally write back only upper or lower 8 bits.	
Argument	c: Either A or B, indicate counter to use ctr_add: 16-bit value to add to counter If opcode is ADDcH, only most significant 8 bits are written back If opcode is ADDcL, only least-significant 8 bits are written back If opcode is ADDc, all 16 bits will be written back (h=1, l=1).
Pseudo-code	<pre> tmp <= ctr_c + ctr_add if (h) begin ctr_c[15:8] <= tmp[15:8] end if (l) begin ctr_c[7:0] <= tmp[7:0] end IP <= IP + 1 </pre>
Note	The NOP (no operation) pseudo-op is encoded as 'ADDA 0'.

Table 59.4-7. IF Opcode

IF ctl_cond_src, tgt	
Jump to target if the indicated mux source bit is set.	
Argument	ctl_cond_src: Mux source (see CTR_MUX_SRC sources) tgt: 3-bit target instruction addresses
Pseudo-code	<pre> if (ctl_cond_src) begin IP <= tgt end else begin IP <= IP + 1 end </pre>
Note	The JMP (always jump) pseudo-op is implemented as an 'IF 95,tgt' opcode, using the always-1 bit from the AUX bits.

Table 59.4-8. IFN Opcode

IFN ctl_cond_src, tgt	
Jump to target if the indicated mux source bit is NOT set (note: same as IF but condition is negated).	
Argument	ctl_cond_src: Mux source (see CTR_MUX_SRC sources) tgt: 3-bit target instruction addresses
Pseudo-code	<pre> if (ctl_cond_src) begin IP <= IP + 1 end else begin IP <= tgt end </pre>

Table 59.4-9. LDCTD Opcode

LDCTDc[H L] ctr_set	
Load counter direct (from immediate), and optionally only load high or low 8 bits of counter.	
Argument	c: Either A or B, indicate counter to use ctr_set: 16-bit value to set the counter to h: if opcode is LDCTDcH, only most significant 8 bits are set l: if opcode is LDCTDcL, only least-significant 8 bits are set If opcode is LDCTDc, all 16 bits are set (h=1, l=1)
Pseudo-code	<pre> if (h) begin ctr_c[15:8] <= ctr_set[15:8] end if (l) begin ctr_c[7:0] <= ctr_set[7:0] end IP <= IP + 1 </pre>

Table 59.4-10. LDCTI Opcode

LDCTIc[H L]	
Load counter indirect (from most significant 16 bits of mux output), and optionally only load high or low 8 bits of counter.	
Argument	c: Either A or B, indicate counter to use h: If opcode is LDCTIcH, only the most significant 8 bits are set l: If opcode is LDCTIcL, only the least significant 8 bits are set If opcode is LDCTIc, all 16 bits are set (h=1, l=1)
Pseudo-code	<pre> if (h) begin ctr_c[15:8] <= mux_out[31:24] end if (l) begin ctr_c[7:0] <= mux_out[23:16] end IP <= IP + 1 </pre>

59.4.2 Configuration Registers

The BitScrambler is configured using a set of registers. Specifically, these registers are used to write a program into BitScrambler, write its LUT RAM, configure its behaviour, and start and stop BitScrambler operation. They are also used to attach a BitScrambler to a peripheral DMA path.

The ESP32-P4 has two BitScramblers: one can be placed in the TX path (where data is sent from the memory to a peripheral) while the other can be placed in a RX path (where data is sent from the peripheral to memory). The configuration registers for the two BitScramblers are the same, with the only difference being the TX path BitScrambler registers are prefixed by BITSCRAMBLER_TX_ while the RX path BitScrambler uses BITSCRAMBLER_RX_ prefixes.

Note:

For convenience, in this section we will refer to the TX path BitScrambler only; to get the RX path BitScrambler descriptions, simply swap the prefixes.

When modifying the input or output of a peripheral, the BitScrambler needs to be attached to the DMA path of that peripheral. To do this, write the appropriate value for the peripheral into the `HP_SYSTEM_BITSCRAMBLER_PERI_TX_SEL` or `HP_SYSTEM_BITSCRAMBLER_PERI_RX_SEL` field of the `HP_SYSTEM_BITSCRAMBLER_PERI_SEL_REG` register.

The BitScrambler can also run in memory-to-memory mode. For this, the `BITSCRAMBLER_LOOP_MODE` bit in `BITSCRAMBLER_SYS_REG` needs to be set to one. In this mode, the TX BitScrambler is put into loopback mode while the RX BitScrambler is unused and unavailable. The BitScrambler still needs to be attached to a peripheral (although the peripheral does not need to be configured); the DMA path for this peripheral will be unusable.

Neither the program memory nor the LUT RAM are accessible to the main processor directly. Rather, they need to be written indirectly, by setting an address in one register and then writing the data in a second register.

Specifically, for the LUT RAM, the address is written in the `BITSCRAMBLER_TX_LUT_IDX` field of `BITSCRAMBLER_TX_LUT_CFG0_REG` and the data is written to or read from `BITSCRAMBLER_TX_LUT_CFG1_REG`. The width of the LUT, as visible to the BitScrambler, can be configured using the `BITSCRAMBLER_TX_LUT_MODE` field. Note that when the host processor writes to the LUT in the aforementioned fashion, the addressing and data size needs to adhere to the configured LUT width.

Similarly, the instruction memory is written by setting the address up in `BITSCRAMBLER_TX_INST_CFG0_REG` and writing the data to or reading it from `BITSCRAMBLER_TX_INST_CFG1_REG`. As the BitScrambler instructions are 257 bits, one instruction needs to be written as 9 32-bit words (with the 257th bit being in the LSB of the 9th word). To achieve that purpose, the `BITSCRAMBLER_TX_INST_CFG0_REG` register is divided into two fields. The position of the instruction is written into the `BITSCRAMBLER_TX_INST_IDX` field while the word offset within the instruction is written to `BITSCRAMBLER_TX_INST_POS`.

As the BitScrambler has the ability to generate more or less data than it gets on the input, a BitScrambler-controlled DMA stream can end in one or two ways. The first is that the receiver (either memory in case of the RX BitScrambler, or the peripheral in case of the TX BitScrambler) stops accepting data because it is configured to only receive a limited number of bytes. This case is pretty simple: the BitScrambler will simply halt as it cannot write any more data. The other way is that the transmitter (the peripheral for the RX BitScrambler and the memory for the TX BitScrambler) stops sending data. This is a more complicated situation, as the BitScrambler may or may not be processing some data that needs to be written to the output.

In order to allow the BitScrambler to send out the data it is still processing, two methods have been devised to handle a certain amount of data after the input data stream has ended. Both depend on setting `BITSCRAMBLER_RX_TAILING_BITS_REG` to a certain value N:

- EOF-on-N-reads: After the input data stream ends, the BitScrambler reads N dummy bytes from the input. The value of these bytes is zero. After the Nth byte is read, the BitScrambler halts and the DMA stream ends.
- EOF-on-N-writes: After the input data stream ends, the BitScrambler is allowed to write N more bytes to

the output. During this time, any reads on the input stream return zero-value dummy bytes. After the N'th byte is written, the BitScrambler halts and the DMA stream ends.

To configure either mode, set the option bits as follows:

Table 59.4-11. Settings for EOF modes

Register	EOF-on-N-reads	EOF-on-N-writes
BITSCRAMBLER_TX_RD_DUMMY	1	1
BITSCRAMBLER_TX_FETCH_MODE	0	0
BITSCRAMBLER_TX_EOF_MODE	1	0
BITSCRAMBLER_TX_HALT_MODE	1	1

59.5 Programming Procedures

Note that these instructions are written for a TX BitScrambler, which is in the memory-to-peripheral path, or memory-to-memory path if in loopback mode. For the peripheral-to-memory path, the RX BitScrambler is used. Unless otherwise indicated, you can exchange "TX" for "RX" in these instructions to configure the RX BitScrambler.

1. Attach the BitScrambler to a peripheral by configuring [HP_SYSTEM_BITSCRAMBLER_PERI_TX_SEL](#) field in [HP_SYSTEM_BITSCRAMBLER_PERI_SEL_REG](#) register. Note that this step is required even in loopback mode.
2. Enable the BitScrambler by setting the [BITSCRAMBLER_TX_ENA](#) field in the [BITSCRAMBLER_TX_CTRL_REG](#) register.
3. Configure loopback mode if required. If loopback mode is needed, set [BITSCRAMBLER_LOOP_MODE](#) in the [BITSCRAMBLER_SYS_REG](#) register. Otherwise, clear it.
4. Load program into BitScrambler. For each instruction and each 32-bit word within the instruction, set the [BITSCRAMBLER_TX_INST_IDX](#) and [BITSCRAMBLER_TX_INST_POS](#) fields in the [BITSCRAMBLER_TX_INST_CFG0_REG](#) register, then write the word in the [BITSCRAMBLER_TX_INST_CFG1_REG](#) register. Note that the 257th bit is written as bit 0 into [BITSCRAMBLER_TX_INST_CFG1_REG](#).
5. Load LUT into BitScrambler, if needed:
 - (a) Configure [BITSCRAMBLER_TX_LUT_MODE](#) in [BITSCRAMBLER_TX_LUT_CFG0_REG](#) to set the LUT width
 - (b) Write the address to the [BITSCRAMBLER_TX_LUT_CFG0_REG](#)
 - (c) Write the data word or the byte itself to [BITSCRAMBLER_TX_LUT_CFG1_REG](#)

Note that it is acceptable to change the LUT width after writing the data and before starting the BitScrambler.

6. Configure BitScrambler. Dependent on what the BitScrambler program expects, you may need to set or clear the various bits in the [BITSCRAMBLER_TX_CTRL_REG](#) register. You may also need to set [BITSCRAMBLER_TX_TAILING_BITS_REG](#) to an applicable value.

7. Start the BitScrambler by clearing the [BITSCRAMBLER_TX_HALT](#) bit in the [BITSCRAMBLER_TX_CTRL_REG](#) register.
8. Start DMA transaction. Please refer to Chapter 4 *GDMA Controller (GDMA-AHB, GDMA-AXI)* for more information about DMA subsystem and refer to the peripheral chapter if loopback mode is not used.
9. Wait until DMA transaction is done.
10. Halt the BitScrambler by setting the [BITSCRAMBLER_TX_HALT](#) in the [BITSCRAMBLER_TX_CTRL_REG](#) register.
11. Wait for halt state. The BitScrambler is halted when the [BITSCRAMBLER_TX_IN_IDLE](#) bit in the [BITSCRAMBLER_TX_STATE_REG](#) register is set.
12. Reset the FIFO by writing a 1 and then a 0 to the [BITSCRAMBLER_TX_FIFO_RST](#) bit in the [BITSCRAMBLER_TX_CTRL_REG](#) register.
13. The BitScrambler is ready for a new transaction with the current program and LUT configuration. Simply start from step 7 to do this.

59.6 Register Summary

The addresses in this section are relative to the Bit-scrambler base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section [Access Types for Registers](#).

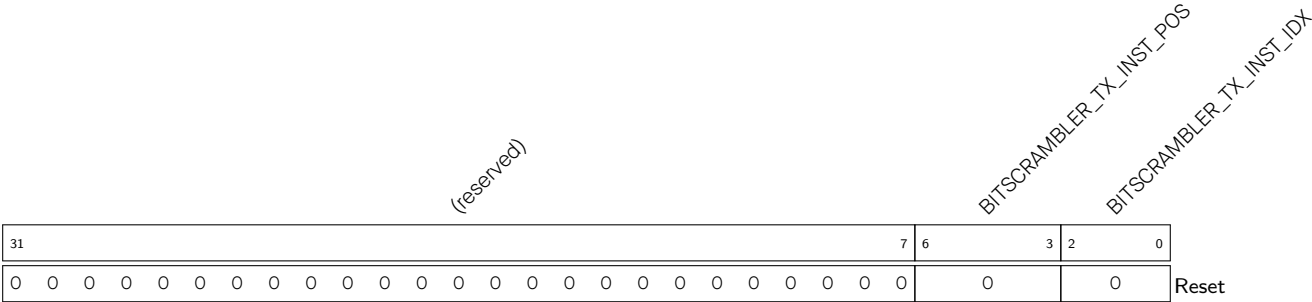
Name	Description	Address	Access
Control and configuration registers			
BITSRAMBLER_TX_INST_CFG0_REG	BitScrambler TX core instruction memory address selection register	0x0000	R/W
BITSRAMBLER_TX_INST_CFG1_REG	BitScrambler TX core instruction memory access register	0x0004	R/W
BITSRAMBLER_RX_INST_CFG0_REG	BitScrambler RX core instruction memory address selection register	0x0008	R/W
BITSRAMBLER_RX_INST_CFG1_REG	BitScrambler RX core instruction memory access register	0x000C	R/W
BITSRAMBLER_TX_LUT_CFG0_REG	BitScrambler TX core LUT memory address selection register	0x0010	R/W
BITSRAMBLER_TX_LUT_CFG1_REG	BitScrambler TX core LUT memory access register	0x0014	R/W
BITSRAMBLER_RX_LUT_CFG0_REG	BitScrambler RX core LUT memory address selection register	0x0018	R/W
BITSRAMBLER_RX_LUT_CFG1_REG	BitScrambler RX core LUT memory access register	0x001C	R/W
Configuration registers			
BITSRAMBLER_TX_TAILING_BITS_REG	BitScrambler TX core extra data length register	0x0020	R/W
BITSRAMBLER_RX_TAILING_BITS_REG	BitScrambler RX core extra data length register	0x0024	R/W
BITSRAMBLER_TX_CTRL_REG	BitScrambler TX core control register	0x0028	varies
BITSRAMBLER_RX_CTRL_REG	BitScrambler RX core control register	0x002C	varies
BITSRAMBLER_SYS_REG	Loopback control register	0x00F8	R/W
Status registers			
BITSRAMBLER_TX_STATE_REG	BitScrambler TX core status register	0x0030	varies
BITSRAMBLER_RX_STATE_REG	BitScrambler RX core status register	0x0034	varies
Version register			
BITSRAMBLER_VERSION_REG	Version control register	0x00FC	R/W

59.7 Registers

The addresses in this section are relative to the Bit-scrambler base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

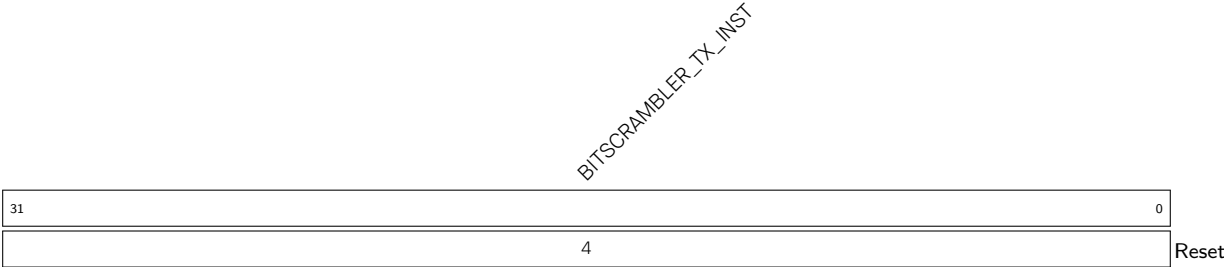
Register 59.1. BITSCRAMBLER_TX_INST_CFG0_REG (0x0000)



BITSCRAMBLER_TX_INST_IDX Configures the index of the BitScrambler instruction to be accessed via [BITSCRAMBLER_TX_INST_CFG1_REG](#). (R/W)

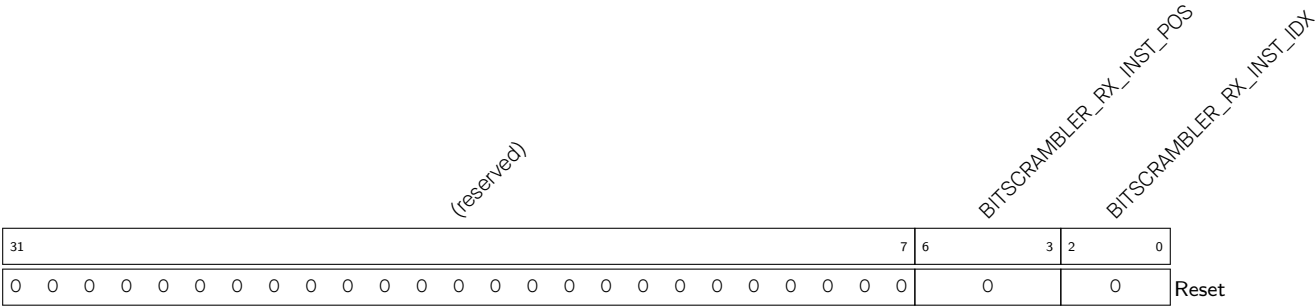
BITSCRAMBLER_TX_INST_POS Configures the offset into the 257-bit BitScrambler instruction (in 32-bit increments) to be accessed via [BITSCRAMBLER_TX_INST_CFG1_REG](#). (R/W)

Register 59.2. BITSCRAMBLER_TX_INST_CFG1_REG (0x0004)



BITSCRAMBLER_TX_INST Configures the instruction to be accessed at the address specified by [BITSCRAMBLER_TX_INST_CFG0_REG](#). (R/W)

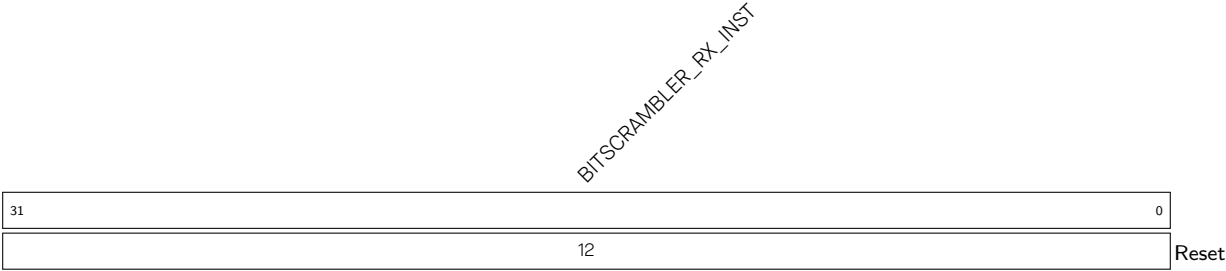
Register 59.3. BITSCRAMBLER_RX_INST_CFG0_REG (0x0008)



BITSCRAMBLER_RX_INST_IDX Configures the index of the BitScrambler instruction to be accessed via [BITSCRAMBLER_RX_INST_CFG1_REG](#). (R/W)

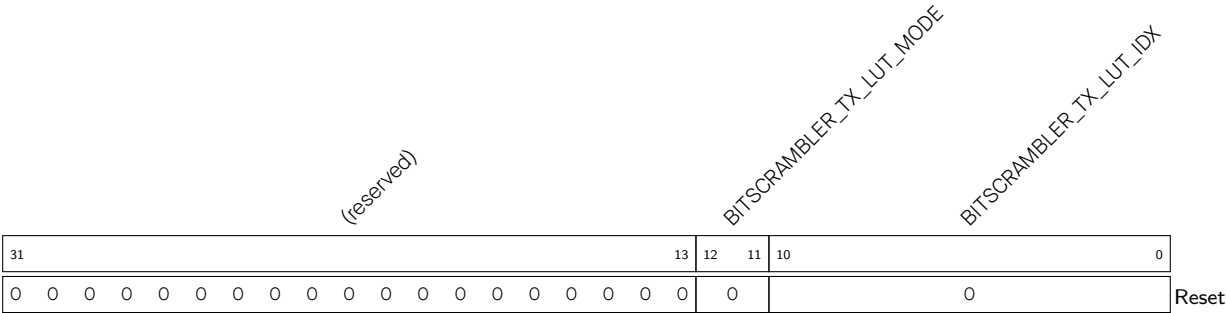
BITSCRAMBLER_RX_INST_POS Configures the offset into the 257-bit BitScrambler instruction (in 32-bit increments) to be accessed via [BITSCRAMBLER_RX_INST_CFG1_REG](#). (R/W)

Register 59.4. BITSCRAMBLER_RX_INST_CFG1_REG (0x000C)



BITSCRAMBLER_RX_INST Configures the instruction to be accessed at the address specified by [BITSCRAMBLER_RX_INST_CFG0_REG](#). (R/W)

Register 59.5. BITSCRAMBLER_TX_LUT_CFG0_REG (0x0010)



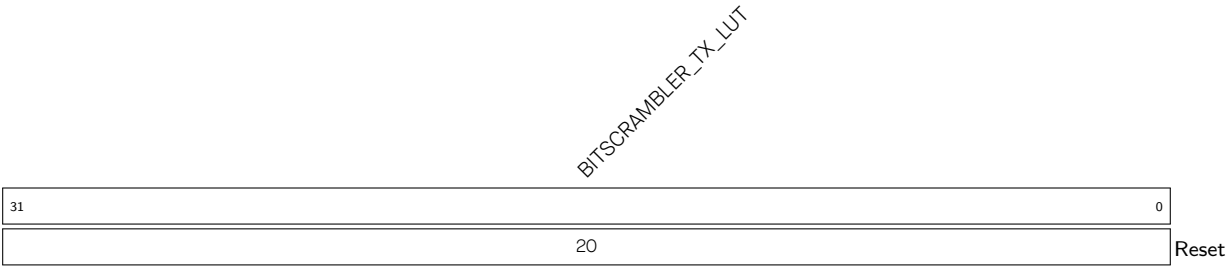
BITSCRAMBLER_TX_LUT_IDX Configures where in LUT RAM to access. Measurement unit: word size configured by [BITSCRAMBLER_TX_LUT_MODE](#). (R/W)

BITSCRAMBLER_TX_LUT_MODE Configures the word size of LUT RAM for BitScrambler programs.

- 0: 1 Byte
- 1: 2 Bytes
- 2: 4 Bytes
- 3: Reserved

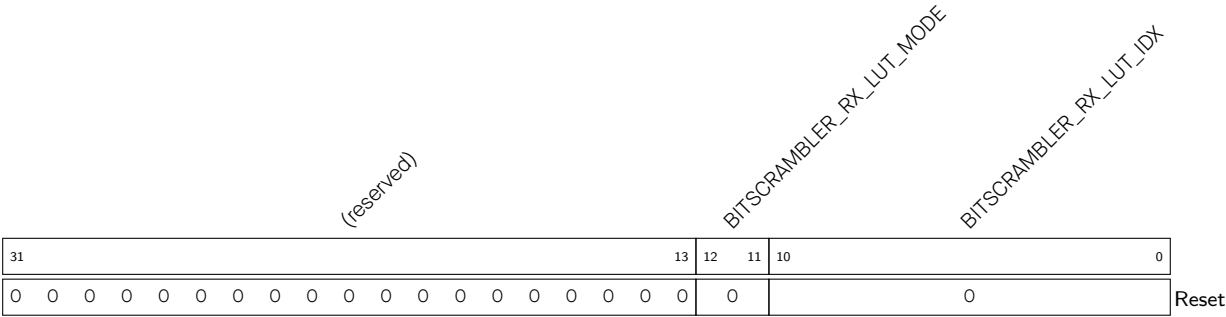
(R/W)

Register 59.6. BITSCRAMBLER_TX_LUT_CFG1_REG (0x0014)



BITSCRAMBLER_TX_LUT Configures the LUT entry to be accessed at the position specified by [BITSCRAMBLER_TX_LUT_CFG0_REG](#). (R/W)

Register 59.7. BITSCRAMBLER_RX_LUT_CFG0_REG (0x0018)



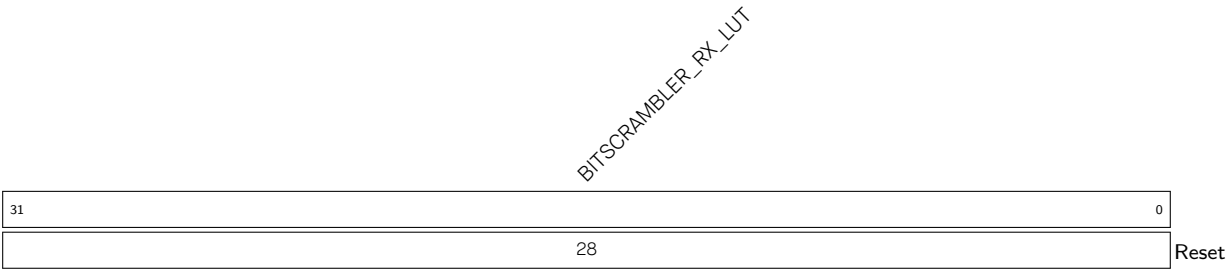
BITSCRAMBLER_RX_LUT_IDX Configures where in LUT RAM to access. Measurement unit: word size configured by [BITSCRAMBLER_RX_LUT_MODE](#). (R/W)

BITSCRAMBLER_RX_LUT_MODE Configures the word size of LUT RAM for BitScrambler programs.

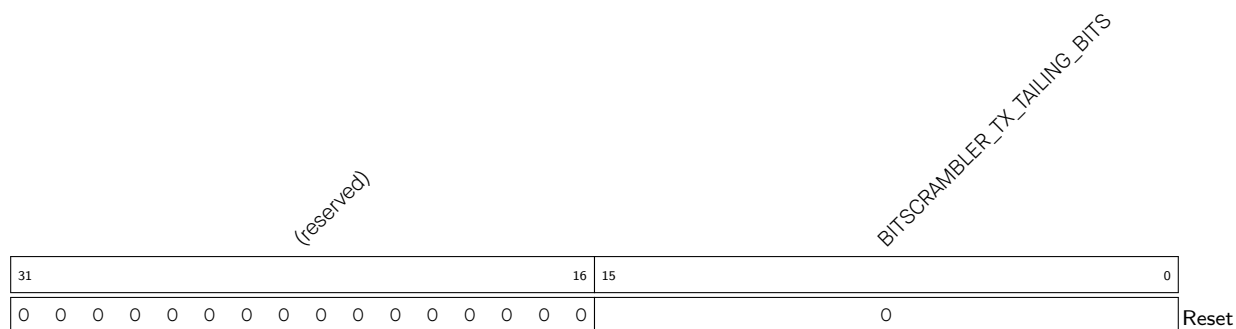
- 0: 1 byte
- 1: 2 bytes
- 2: 4 bytes
- 3: Reserved

(R/W)

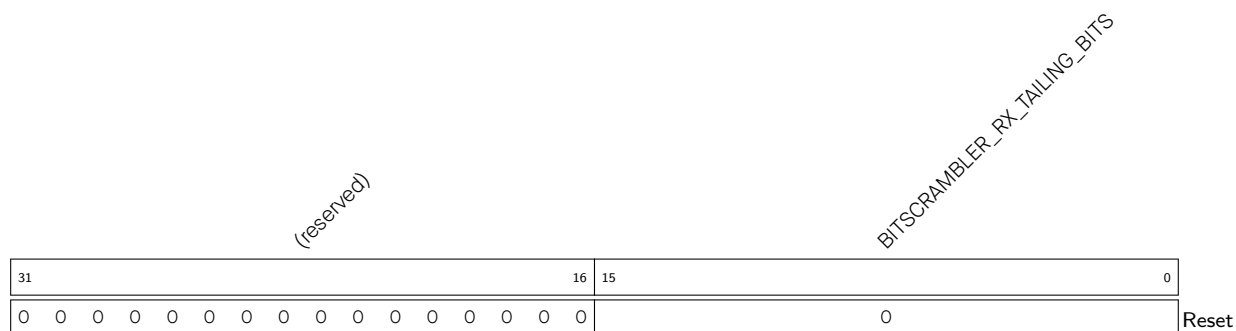
Register 59.8. BITSCRAMBLER_RX_LUT_CFG1_REG (0x001C)



BITSCRAMBLER_RX_LUT Configures the LUT entry to be accessed at the position specified by [BITSCRAMBLER_RX_LUT_CFG0_REG](#). (R/W)

Register 59.9. BITSCRAMBLER_TX_TAILING_BITS_REG (0x0020)

BITSCRAMBLER_TX_TAILING_BITS Configures the length of extra data after getting EOF for the TX BitScrambler core. Measurement unit: bit. (R/W)

Register 59.10. BITSCRAMBLER_RX_TAILING_BITS_REG (0x0024)

BITSCRAMBLER_RX_TAILING_BITS Configures the length of extra data after getting EOF for the BitScrambler RX core. Measurement unit: bit. (R/W)

Register 59.11. BITSCRAMBLER_TX_CTRL_REG (0x0028)

(reserved)																								BITSCRAMBLER_TX_FIFO_RST BITSCRAMBLER_TX_RD_DUMMY BITSCRAMBLER_TX_HALT_MODE BITSCRAMBLER_TX_FETCH_MODE BITSCRAMBLER_TX_COND_MODE BITSCRAMBLER_TX_EOF_MODE BITSCRAMBLER_TX_PAUSE BITSCRAMBLER_TX_ENA																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
31																								9	8	7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

BITSCRAMBLER_TX_ENA Configures whether to enable BitScrambler TX core.

0: Disable

1: Enable

(R/W)

BITSCRAMBLER_TX_PAUSE Configures whether to pause BitScrambler TX core. A paused core can be un-paused to resume execution.

0: Not pause

1: Pause

(R/W)

BITSCRAMBLER_TX_HALT Configures whether to halt BitScrambler TX core. Halting the BitScrambler TX core will flush the FIFOs and cannot be resumed; a FIFO reset and a restart is needed.

0: Not halt

1: Halt (R/W)

BITSCRAMBLER_TX_EOF_MODE Configures BitScrambler TX core EOF signal generating mode.

This is combined with [BITSCRAMBLER_TX_TAILING_BITS](#) for use.

0: Data written to the output FIFO are counted to generate delayed EOF

1: Data read from the input FIFO are counted to generate delayed EOF

(R/W)

BITSCRAMBLER_TX_COND_MODE Configures BitScrambler TX LOOP instruction condition mode.

0: Use the less than operator to get the condition

1: Use not equal operator to get the condition

(R/W)

BITSCRAMBLER_TX_FETCH_MODE Configures BitScrambler TX core fetch instruction mode.

0: Prefetch by reset

1: Fetch by instructions

(R/W)

BITSCRAMBLER_TX_HALT_MODE Configures BitScrambler TX core halt mode when [BITSCRAMBLER_TX_HALT](#) is set

0: Wait for write data back done

1: Ignore write data back

(R/W)

Continued on the next page...

Register 59.11. BITSCRAMBLER_TX_CTRL_REG (0x0028)

Continued from the previous page...

BITSCRAMBLER_TX_RD_DUMMY Configures BitScrambler TX core read data mode when EOF received.

0: Wait read data

1: Ignore read data

(R/W)

BITSCRAMBLER_TX_FIFO_RST Configures whether to reset BitScrambler TX FIFO.

0: Not reset

1: Reset

(WT)

Register 59.12. BITSCRAMBLER_RX_CTRL_REG (0x002C)

(reserved)																								BITSCRAMBLER_RX_FIFO_RST BITSCRAMBLER_RX_RD_DUMMY BITSCRAMBLER_RX_HALT_MODE BITSCRAMBLER_RX_FETCH_MODE BITSCRAMBLER_RX_COND_MODE BITSCRAMBLER_RX_EOF_MODE BITSCRAMBLER_RX_PAUSE BITSCRAMBLER_RX_ENA										
31																								9	8	7	6	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0							

BITSCRAMBLER_RX_ENA Configures whether to enable BitScrambler RX core.

0: Disable

1: Enable

(R/W)

BITSCRAMBLER_RX_PAUSE Configures whether to pause BitScrambler RX core. A paused core can be un-paused to resume execution.

0: Not pause

1: Pause

(R/W)

BITSCRAMBLER_RX_HALT Configures whether to halt BitScrambler RX core. Halting the BitScrambler RX core will flush the FIFOs and cannot be resumed; a FIFO reset and a restart is needed.

0: Not halt

1: Halt

(R/W)

BITSCRAMBLER_RX_EOF_MODE Configures BitScrambler RX core EOF signal generating mode.

This is combined with [BITSCRAMBLER_RX_TAILING_BITS](#) for use.

0: Data written to the output FIFO are counted to generate delayed EOF

1: Data read from the input FIFO are counted to generate delayed EOF

(R/W)

BITSCRAMBLER_RX_COND_MODE Configures BitScrambler RX LOOP instruction condition mode.

0: Use the less than operator to get the condition

1: Use not equal operator to get the condition

(R/W)

BITSCRAMBLER_RX_FETCH_MODE Configures BitScrambler RX core fetch instruction mode

0: Prefetch by reset

1: Fetch by instructions

(R/W)

Continued on the next page...

Register 59.12. BITSCRAMBLER_RX_CTRL_REG (0x002C)

Continued from the previous page...

BITSCRAMBLER_RX_HALT_MODE Configures BitScrambler RX core halt mode when **BITSCRAMBLER_RX_HALT** is set

0: Wait for write data back done

1: Ignore write data back

(R/W)

BITSCRAMBLER_RX_RD_DUMMY Configures BitScrambler RX core read data mode when EOF received.

0: Wait read data

1: Ignore read data

(R/W)

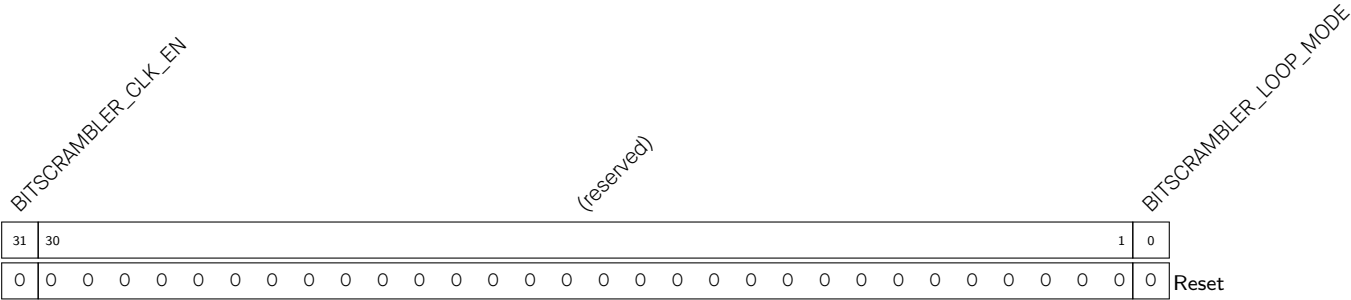
BITSCRAMBLER_RX_FIFO_RST Configures whether to reset BitScrambler RX FIFO.

0: Not reset

1: Reset

(WT)

Register 59.13. BITSCRAMBLER_SYS_REG (0x00F8)



BITSCRAMBLER_LOOP_MODE Configures whether to enable BitScrambler TX to DMA RX loopback mode

0: Disable

1: Enable

(R/W)

BITSCRAMBLER_CLK_EN Reserved. (R/W)

Register 59.14. BITSCRAMBLER_TX_STATE_REG (0x0030)

31	30	29	16	15	5	4	3	2	1	0	Reset
0	0	0	0	0	0	0	0	0	0	1	0x00000001

BITSCRAMBLER_TX_IN_IDLE Represents whether BitScrambler TX core is halted.

0: Not halted

1: Halted

(RO)

BITSCRAMBLER_TX_IN_RUN Represents whether BitScrambler TX core is running.

0: Not running

1: Running

(RO)

BITSCRAMBLER_TX_IN_WAIT Represents whether BitScrambler TX core is waiting for write back done.

0: Not waiting

1: Waiting

(RO)

BITSCRAMBLER_TX_IN_PAUSE Represents whether BitScrambler TX core is paused.

0: Not paused

1: Paused

(RO)

BITSCRAMBLER_TX_FIFO_EMPTY Represents whether BitScrambler TX FIFO is empty

0: Not empty

1: Empty

(RO)

BITSCRAMBLER_TX_EOF_GET_CNT	Represents byte count of BitScrambler TX core after EOF is received. (RO)
------------------------------------	---

SCRAMBLER_TX_EOF_OVERLOAD Represents whether BitScrambler TX core tries to process more than one EOF.

0: Not try to process more than one EOF

1: Try to process more than one EOF

(RO)

BITSCRAMBLER_TX_EOF_TRACE_CLR Configures whether to clear **BITSCRAMBLER_TX_EOF_OVERLOAD** and **BITSCRAMBLER_TX_EOF_GET_CNT**.

0: Not clear

1: Clear

(WT)

Register 59.15. BITSCRAMBLER_RX_STATE_REG (0x0034)

BITSCRAMBLER_RX_EOF_TRACE_CLR BITSCRAMBLER_RX_EOF_OVERLOAD			BITSCRAMBLER_RX_EOF_GET_CNT												(reserved)					BITSCRAMBLER_RX_FIFO_FULL BITSCRAMBLER_RX_IN_PAUSE BITSCRAMBLER_RX_IN_WAIT BITSCRAMBLER_RX_IN_RUN BITSCRAMBLER_RX_IN_IDLE										
31	30	29	16												15	5					4	3	2	1	0					
0	0	0												0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	Reset

BITSCRAMBLER_RX_IN_IDLE Represents whether BitScrambler TX core is halted.

0: Not halted

1: Halted

(RO)

BITSCRAMBLER_RX_IN_RUN Represents whether BitScrambler TX core is running.

0: Not running

1: Running

(RO)

BITSCRAMBLER_RX_IN_WAIT Represents whether BitScrambler TX core is waiting for write back done.

0: Not waiting

1: Waiting

(RO)

BITSCRAMBLER_RX_IN_PAUSE Represents whether BitScrambler TX core is paused.

0: Not paused

1: Paused

(RO)

BITSCRAMBLER_RX_FIFO_FULL Represents whether BitScrambler RX FIFO is full.

0: Not full

1: Full

(RO)

BITSCRAMBLER_RX_EOF_GET_CNT Represents byte count of BitScrambler TX core after EOF is received. (RO)

BITSCRAMBLER_RX_EOF_OVERLOAD Represents whether BitScrambler RX core tries to process more than one EOF.

0: Not try to process more than one EOF

1: Try to process more than one EOF

(RO)

BITSCRAMBLER_RX_EOF_TRACE_CLR Configures whether to clear [BITSCRAMBLER_RX_EOF_OVERLOAD](#) and [BITSCRAMBLER_RX_EOF_GET_CNT](#).

0: Not clear

1: Clear

(WT)

Register 59.16. BITSCRAMBLER_VERSION_REG (0x00FC)



BITSCRAMBLER_BITSCRAMBLER_VER Version control register. (R/W)